waterX: Smart Plant Water Management System

Project Report, Requirements and Database Design

October 17, 2026

Contents

1 Introduction **1**

2 Project Implementation **1**
- 2.1 Architecture and Features . . . 1
- 2.2 Frontend Dashboard . . . 1
  - 2.2.1 Dashboard overview . . . 1
  - 2.2.2 Plant management and detail screens . . . 3
  - 2.2.3 Settings, persistence and demonstration data . . . 3
  - 2.2.4 Navigation scope . . . 3
- 2.3 Simulation Behavior . . . 3
- 2.4 Running the Project Manually . . . 4

3 Functional Requirements **4**

4 Non-Functional Requirements **5**

5 Database Design **6**
- 5.1 Overview . . . 6
- 5.2 ER Diagram . . . 6
- 5.3 Relationships . . . 7
- 5.4 Data Dictionary . . . 7
- 5.5 Indexes and Views . . . 11

1 Introduction

waterX is a smart plant-water-management project with two related but currently separate parts: a browser-based demonstration dashboard and a Python/SQLite simulation backend. The simulator generates weather and soil conditions, applies watering strategies and safety limits, and stores run data. The dashboard demonstrates plant management and monitoring screens using sample data stored in the browser; it is not connected to the simulator database or to physical sensors.

Research question. Can AI-assisted watering reduce water consumption compared with fixed-schedule watering without negatively affecting plant growth?

This report describes the project structure, implemented behavior, known limitations, requirements and SQLite database design. The simulator's AI-assisted strategy currently uses a physics-based moisture forecast; a trained machine-learning model and a measured strategy-comparison study have not been implemented. Requirements use the MoSCoW priority scale (*Must*, *Should*, *Could*); implementation status is shown separately.

2 Project Implementation

2.1 Architecture and Features

The project is organized into a Next.js frontend, a Python simulation engine and a SQLite database. The two applications are not wired together: browser plant and preference data is saved in `localStorage`, while simulator and seed data is saved in `database/waterx.db`.

Project area	Implemented features	Important boundary
Frontend	Responsive dashboard with plant-status cards, sample moisture and tank charts, sample predictions, alerts, recent watering events and quick-action dialogs.	Dashboard metrics, charts, alerts, events and recommendations are explicitly demonstration data; they are not live simulator output.
Plant management	Add, edit, search, sort, filter, deactivate, reactivate and delete plants. Plant details include moisture history, watering history, growth, prediction and care settings panels.	Changes persist in this browser's `localStorage`; new plants have no sensor readings.
Preferences	Change the displayed profile name and light/dark appearance; reset preferences to defaults.	Preferences are local to the browser and are not user-account settings.
Simulator	Hourly weather and moisture simulation; fixed-schedule, threshold-based and forecast-based watering; noisy/faulty sensor readings; tank, daily-cycle and pump-runtime safeguards; alerts and simulated growth.	The forecast is a deterministic physics-based heuristic, not a trained ML model.
Database	SQLite schema, repeatable demonstration seed data, foreign-key enforcement, integrity constraints, indexes and a daily-water-usage view.	The frontend does not currently read from or write to this database.
Tests	Python unit tests exercise database setup, seed repeatability, all watering strategies, data persistence, sensor/tank/pump safeguards and seeded reproducibility.	These tests do not establish frontend/backend integration or model accuracy.

Table 1: Implemented project components and their current boundaries.

2.2 Frontend Dashboard

The frontend is a client-side Next.js application built with React and TypeScript. Recharts supplies the dashboard charts, and Lucide React supplies the interface icons. Its visual design combines a dark-blue navigation sidebar, a top search/profile bar, a light dashboard canvas and separate white metric and content cards. The layout reorganizes for narrower screens. Figure 1 shows the dashboard design included in the project.

2.2.1 Dashboard overview

The home route `/` is the main monitoring screen. It presents four summary cards for plant count and health, tank level, weekly water use and displayed prediction accuracy. Below these cards, the plant-status section shows each plant's category, moisture, temperature, tank level, status

and next-watering text. The right-hand activity column groups alerts and notifications, recent watering events and quick actions. The dashboard also contains soil-moisture trend lines with a period selector, a circular tank-level chart, a six-hour observed-versus-predicted moisture chart, and a recommendation panel.

The top bar provides plant search, a notification button and the saved display name. The Add Plant quick action opens a form. Other dashboard actions open dialogs for notifications, watering history, reports, schedules or the recommendation. The report dialog summarizes sample values; schedule editing and database-backed reporting are not implemented.

2.2.2 Plant management and detail screens

The `/plants` route is the plant library. Its summary cards show total and active plants, healthy plants, plants needing attention and average available moisture. The library supports searching by plant name, species, category or ID, filtering by status, and sorting by name, moisture or last-watered time. Plant cards expose view, edit, deactivate/reactivate and delete actions. Add and edit forms configure name, species, optional image URL, minimum/optimal/ maximum moisture, water amount, daily cycle limit, maximum pump runtime and schedule text. Validation enforces ordered moisture thresholds. The confirmation dialogs explain the effect of deactivation and deletion.

Each `/plants/<plantId>` detail screen shows an overview and configured care information, a selectable 24-hour/7-day/30-day moisture-history chart with threshold references, watering history, growth measurements and a demonstration moisture outlook. Its plant-specific settings form can update care values. Empty and unavailable states are shown when readings or growth records do not exist. A newly created plant intentionally starts without a sensor reading, so it is marked as having a sensor error until readings become available.

2.2.3 Settings, persistence and demonstration data

The `/settings` route allows the user to change the displayed name, select light or dark appearance, and restore preference defaults. Plant records and settings are persisted separately in browser `localStorage`; they belong to that browser profile, not to a server-side account. Invalid saved plant data and storage write failures are surfaced in the interface.

The dashboard starts with sample plants and prewritten chart, alert, watering-event and recommendation data. Demo labels and explanatory notes identify sample values; in particular, the accuracy percentage and moisture prediction are illustrative and are not measured model results. The frontend does not yet consume the simulator's SQLite records or connect to real sensors, a pump, or a backend API.

2.2.4 Navigation scope

The sidebar includes Dashboard, My Plants, Sensor Data, Watering Schedule, AI Predictions, Growth Monitoring, Reports and Settings. Dashboard, My Plants and Settings have working routes; plant details are available from My Plants. The other sidebar sections currently display a placeholder explaining that the section is not implemented. Likewise, schedule editing, database-backed reports and CSV export are not part of the current frontend.

2.3 Simulation Behavior

The Python engine advances one hour at a time. Weather and sensor randomness are controlled by the configured seed. Plant moisture changes with simulated temperature, humidity, sunlight, rain and plant uptake. The strategies are:

- `fixed_schedule`: water when the configured schedule time is reached (08:00 is used when a profile has no schedule).

- `rule_based`: water when a valid sensor reading falls below the profile's minimum moisture.
- `ai_assisted`: estimate future moisture from the latest valid reading and current conditions, then water if the estimate is below the minimum. This is a physics-based heuristic forecast.

Faulty or missing sensor readings block watering. The engine also enforces the daily cycle limit, limits delivered water by the profile's maximum pump time and available tank level, and writes alerts when applicable. Runs store weather, readings, watering events, tank levels, predictions for `ai_assisted` runs, and periodic simulated growth records.

2.4 Running the Project Manually

Run Python commands from the repository root. Python 3.11 or later is the intended environment; the backend uses the Python standard library. Install Node.js and npm to run the frontend.

```
python -m database.waterx_db
python -m database.seed
python -m simulator --strategy ai_assisted --hours 72 --seed 42
python -m unittest discover -s tests -v
```

The first command initializes the default SQLite database, the second inserts repeatable demonstration data, and the third runs a simulation using the seeded plants. The simulator also accepts `fixed_schedule` and `rule_based`; `python -m simulator -help` lists its options. To use another database file, pass a `-database` path to the seed or simulator command.

The frontend is in the directory named `frountend`. From that directory, install its Node.js dependencies and start the development server:

```
npm install
npm run dev
```

Open `http://localhost:3000` in a browser. The frontend scripts are defined in `frountend/package.json`; `npm run build` creates a production build and `npm run lint` runs ESLint.

To compile this report manually, run the following command twice from the `docs` directory so the table of contents is updated:

```
pdflatex -output-directory=build waterx.tex
```

The output PDF is `docs/build/waterx.pdf`.

3 Functional Requirements

ID	Requirement	Priority	Status
FR-01	The user can add, edit and deactivate plants and assign each plant a plant profile.	Must	Partial
FR-02	The user can configure each profile's watering settings: minimum, maximum and optimal moisture, water per cycle, maximum cycles per day, maximum pump run time and an optional fixed schedule time.	Must	Partial

ID	Requirement	Priority	Status
FR-03	The simulator generates hourly weather (temperature, humidity, sunlight, rain) and soil moisture per plant, modelling evaporation, plant uptake, watering and rain.	Must	Implemented
FR-04	The simulator adds sensor noise and faults (dropouts, spikes) and flags faulty readings.	Should	Implemented
FR-05	The fixed-schedule strategy waters a set amount at a set time every day (baseline for the experiment).	Must	Implemented
FR-06	The rule-based strategy waters a plant when its moisture falls below the profile threshold.	Must	Implemented
FR-07	Safety rules override every strategy: no watering when the tank is empty or the sensor is faulty, and limits on pump run time and waterings per day.	Must	Implemented
FR-08	The AI-assisted strategy uses predicted moisture to water before moisture drops below the threshold.	Must	Partial
FR-09	The system stores every sensor reading, weather record, watering event and tank level in the database, linked to a simulation run.	Must	Implemented
FR-10	The system raises alerts (low tank, too dry, too wet, sensor fault, pump limit, predicted dry) with a severity level.	Should	Partial
FR-11	The system trains and evaluates moisture-prediction models using a time-based split, reports MAE and RMSE, compares them with a naive baseline, and stores model details and predictions.	Must	Partial
FR-12	The system records plant growth (height, leaf count, biomass, stress level) and compares it with watering history.	Should	Partial
FR-13	The dashboard shows plant status, moisture charts, tank level, water consumption and alerts.	Must	Partial
FR-14	The dashboard lets the user choose the strategy, random seed and simulation speed.	Should	Partial
FR-15	Reports show water use per day, plant and run, and compare the three strategies on water used, time in stress and final growth.	Must	Partial
FR-16	The user can export report data as CSV.	Could	Planned

Here, *Implemented* means present in the Python simulation/database code. *Partial* means some behavior exists, but the full requirement is not delivered or is demo-only. *Planned* means no implementation is present. In particular, the browser dashboard and Python simulation currently operate independently.

4 Non-Functional Requirements

These are proposed requirements, not benchmark results or a compatibility certification. Performance, scalability, and model-accuracy targets have not been measured. The maintainability and hardware-portability statements describe design goals; the current project has no separate ML module and no physical sensor or pump integration.

ID	Category	Requirement
NFR-01	Performance	Dashboard pages load within 2 seconds on a normal laptop with 6 months of simulated data.
NFR-02	Performance	Simulating 6 months for 5 plants completes in under 1 minute at full speed.
NFR-03	Safety	The system fails safe: on a sensor fault or missing reading it must not water.
NFR-04	Reproducibility	The same random seed and settings produce identical results.
NFR-05	Data integrity	Foreign keys are enforced, and CHECK and UNIQUE constraints prevent invalid or duplicate rows.
NFR-06	Accuracy	The prediction model has a lower MAE than the naive "same as now" baseline on the test period.
NFR-07	Maintainability	Code is split into modules (simulator, engine, db, ml, dashboard), with unit tests for the decision engine and safety rules.
NFR-08	Portability	Sensor input and pump output sit behind `SensorSource` and `PumpController` interfaces, so ESP32 hardware can replace the simulator without changing the engine.
NFR-09	Usability	The dashboard is readable on desktop and phone browsers and uses clear status colours.
NFR-10	Scalability	Indexes on `(run_id, plant_id, timestamp)` keep queries fast with millions of rows.
NFR-11	Security	The system runs locally and uses parameterised SQL queries and input validation to prevent SQL injection.
NFR-12	Compatibility	Python 3.11 or later, SQLite 3, and current versions of Chrome, Firefox and Edge.

5 Database Design

5.1 Overview

The database is SQLite with 11 tables in six groups.

- **Configuration:** `plant_profiles`, `plants`
- **Experiments:** `simulation_runs`
- **Time-series:** `weather`, `sensor_readings`, `watering_events`, `tank_levels`
- **Growth:** `growth_records`
- **AI:** `ml_models`, `predictions`
- **Monitoring:** `alerts`

Conventions: timestamps are ISO-8601 text in UTC, water is measured in ml and moisture in percent. Foreign keys are enabled with `PRAGMA foreign_keys = ON`.

5.2 ER Diagram

Figure 2 shows the tables with their primary keys (PK) and foreign keys (FK). Every `simulation_runs` row is the hub that ties the time-series tables to one experiment run.

5.3 Relationships

All relationships are one-to-many. The schema has no direct one-to-one relationship. Plants and simulation runs are related many-to-many, which is resolved through the log tables (`sensor_readings`, `watering_events`, `growth_records`, `predictions`); each carries both a `run_id` and a `plant_id`.

Parent (1)	Child (N)	Foreign key	Optional	Meaning
`plant_profiles`	`plants`	`profile_id`	No	A profile defines many plants
`plants`	`sensor_readings`	`plant_id`	No	A plant has many readings
`plants`	`watering_events`	`plant_id`	No	A plant receives many waterings
`plants`	`growth_records`	`plant_id`	No	A plant has many growth records
`plants`	`predictions`	`plant_id`	No	A plant has many predictions
`plants`	`alerts`	`plant_id`	Yes	Some alerts (such as low tank) are not tied to a plant
`simulation_runs`	`weather`	`run_id`	No	A run contains many weather records
`simulation_runs`	`sensor_readings`	`run_id`	No	A run contains many readings
`simulation_runs`	`watering_events`	`run_id`	No	A run contains many waterings
`simulation_runs`	`tank_levels`	`run_id`	No	A run contains many tank logs
`simulation_runs`	`growth_records`	`run_id`	No	A run contains many growth records
`simulation_runs`	`predictions`	`run_id`	No	A run contains many predictions
`simulation_runs`	`alerts`	`run_id`	Yes	An alert may be outside any run
`ml_models`	`simulation_runs`	`model_id`	Yes	Only AI-assisted runs use a model
`ml_models`	`predictions`	`model_id`	No	A model makes many predictions

Table 4: Foreign-key relationships.

5.4 Data Dictionary

`plant_profiles`

Reusable settings for a plant type, such as basil or tomato.

`Column`	Type	Constraints	Description
`profile_id`	INTEGER	PK, autoincrement	Unique profile id
`name`	TEXT	NOT NULL, UNIQUE	Plant type name
`min_moisture`	REAL	NOT NULL, 0–100	Lower moisture limit (%)
`max_moisture`	REAL	NOT NULL, 0–100	Upper moisture limit (%); must exceed `min_moisture`
`optimal_moisture`	REAL	NOT NULL	Target moisture (%)
`water_per_cycle_ml`	REAL	NOT NULL, > 0	Water delivered per watering (ml)
`max_cycles_per_day`	INTEGER	NOT NULL, default 3	Safety limit on waterings per day
`max_pump_seconds`	INTEGER	NOT NULL, default 30	Safety limit on pump run time
`schedule_time`	TEXT	nullable	Fixed-schedule time, such as 07:00
`uptake_rate`	REAL	NOT NULL	Simulator base water-use rate

`plants`

One plant instance that follows a profile.

Column	Type	Constraints	Description
`plant_id`	INTEGER	PK, autoincrement	Unique plant id
`profile_id`	INTEGER	FK to `plant_profiles`, NOT NULL	The profile this plant follows
`name`	TEXT	NOT NULL	Plant name
`location`	TEXT	nullable	Where the plant is
`planted_at`	TEXT	NOT NULL	Planting date
`active`	INTEGER	NOT NULL, default 1	1 = active, 0 = deactivated

`simulation_runs`

One row per simulation run. Same seed with a different strategy gives a fair comparison.

Column	Type	Constraints	Description
`run_id`	INTEGER	PK, autoincrement	Unique run id
`strategy`	TEXT	NOT NULL, CHECK	`fixed_schedule`, `rule_based` or `ai_assisted`
`random_seed`	INTEGER	NOT NULL	Seed for weather and noise
`start_time`	TEXT	NOT NULL	Simulation start
`end_time`	TEXT	NOT NULL	Simulation end
`model_id`	INTEGER	FK to `ml_models`, nullable	Model used (AI strategy only)
`notes`	TEXT	nullable	Free text
`created_at`	TEXT	NOT NULL, default now	Row creation time

`ml_models`

Metadata for a prediction method or model; trained-model metrics are optional.

Column	Type	Constraints	Description
`model_id`	INTEGER	PK, autoincrement	Unique model id
`name`	TEXT	NOT NULL	Model name, such as `random_forest_v1`
`algorithm`	TEXT	NOT NULL	Algorithm used
`horizon_hours`	INTEGER	NOT NULL	How many hours ahead it predicts
`features`	TEXT	NOT NULL	Description or list of input feature names
`train_start, train_end`	TEXT	nullable	Training period
`mae, rmse`	REAL	nullable	Test-set error metrics
`file_path`	TEXT	nullable	Saved model file
`created_at`	TEXT	NOT NULL, default now	Row creation time

`weather`

Simulated weather for each run.

Column	Type	Constraints	Description
`weather_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK to `simulation_runs`, NOT NULL, cascade delete	Owning run
`timestamp`	TEXT	NOT NULL	Time of the record
`temperature_c`	REAL	nullable	Temperature (°C)
`humidity_pct`	REAL	nullable	Humidity (%)
`sunlight`	REAL	nullable	Sunlight level
`rain_mm`	REAL	default 0	Rainfall (mm)
UNIQUE (`run_id`, `timestamp`)			

`sensor_readings`

Moisture readings per plant, as the sensor reports them.

Column	Type	Constraints	Description
`reading_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK, NOT NULL, cascade delete	Owning run
`plant_id`	INTEGER	FK to `plants`, NOT NULL	Plant measured
`timestamp`	TEXT	NOT NULL	Time of the reading
`moisture_pct`	REAL	nullable	Noisy sensor value (%)
`true_moisture`	REAL	nullable	Simulator ground truth (NULL on real hardware)
`is_fault`	INTEGER	NOT NULL, default 0	1 = dropout or spike
UNIQUE (`run_id`, `plant_id`, `timestamp`)			

`watering_events`

Every time the pump waters a plant.

Column	Type	Constraints	Description
`event_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK, NOT NULL, cascade delete	Owning run
`plant_id`	INTEGER	FK, NOT NULL	Plant watered
`timestamp`	TEXT	NOT NULL	Time of watering
`amount_ml`	REAL	NOT NULL, ≥ 0	Water delivered (ml)
`duration_s`	REAL	nullable	Pump run time (s)
`trigger`	TEXT	NOT NULL, CHECK	`schedule`, `threshold`, `prediction` or `manual`
`moisture_before`	REAL	nullable	Moisture before watering
`predicted_moisture`	REAL	nullable	Prediction that triggered it (AI strategy)

`tank_levels`

Water tank level log.

Column	Type	Constraints	Description
`tank_log_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK, NOT NULL, cascade delete	Owning run
`timestamp`	TEXT	NOT NULL	Time of the log
`level_ml`	REAL	NOT NULL	Water left (ml)
`capacity_ml`	REAL	NOT NULL	Tank capacity (ml)
UNIQUE (`run_id`, `timestamp`)			

`growth_records`

Plant growth over time.

Column	Type	Constraints	Description
`growth_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK, NOT NULL, cascade delete	Owning run
`plant_id`	INTEGER	FK, NOT NULL	Plant measured
`timestamp`	TEXT	NOT NULL	Time of the record
`height_cm`	REAL	nullable	Height (cm)
`leaf_count`	INTEGER	nullable	Number of leaves
`biomass_g`	REAL	nullable	Biomass (g)
`stress_level`	REAL	nullable	0 (none) to 1 (severe)
UNIQUE (`run_id`, `plant_id`, `timestamp`)			

`predictions`

Predicted moisture, stored next to the actual value for evaluation.

Column	Type	Constraints	Description
`prediction_id`	INTEGER	PK, autoincrement	Unique id
`run_id`	INTEGER	FK, NOT NULL, cascade delete	Owning run
`model_id`	INTEGER	FK to `ml_models`, NOT NULL	Model that predicted
`plant_id`	INTEGER	FK, NOT NULL	Plant predicted
`made_at`	TEXT	NOT NULL	When the prediction was made
`target_time`	TEXT	NOT NULL	Time being predicted
`predicted_moisture`	REAL	NOT NULL	Predicted moisture (%)
`actual_moisture`	REAL	nullable	Filled in later

alerts

Warnings raised by the system.

Column	Type	Constraints	Description
alert_id	INTEGER	PK, autoincrement	Unique id
run_id	INTEGER	FK, nullable, cascade delete	Related run
plant_id	INTEGER	FK, nullable	Related plant
timestamp	TEXT	NOT NULL	Time of the alert
type	TEXT	NOT NULL, CHECK	low_tank, too_dry, too_wet, sensor_fault, pump_limit or prediction_dry
severity	TEXT	NOT NULL, CHECK	info, warning or critical
message	TEXT	nullable	Description
resolved	INTEGER	NOT NULL, default 0	1 = resolved

5.5 Indexes and Views

Time-series queries filter by run, plant and time, so these indexes exist:

- sensor_readings(run_id, plant_id, timestamp)
- watering_events(run_id, plant_id, timestamp)
- growth_records(run_id, plant_id, timestamp)
- predictions(run_id, plant_id, target_time)
- alerts(run_id, timestamp)

The view daily_water_usage sums watering-event counts and water use per run, plant and day. It is available for future reports; the frontend does not currently query it or provide a strategy-comparison report.

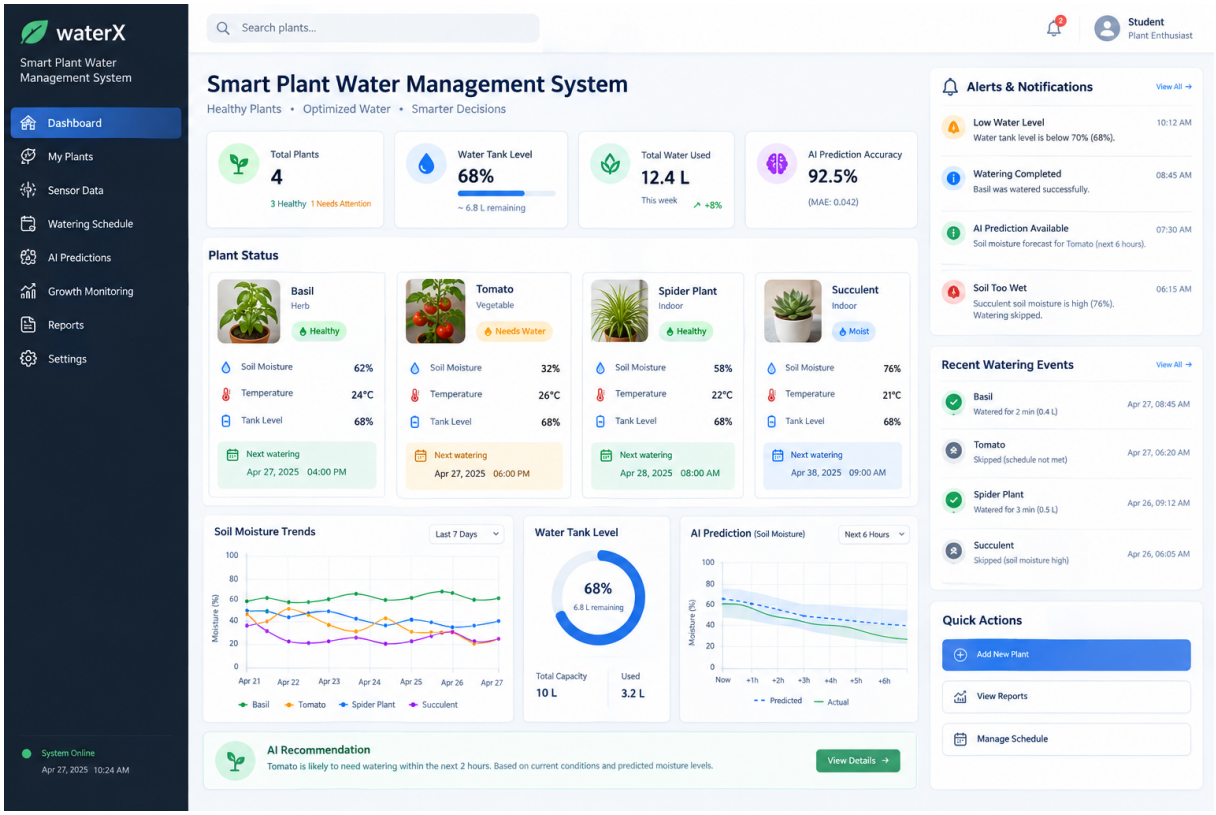


Figure 1: waterX frontend dashboard design. The cards, charts, notifications and recommendations are sample demonstration content, not live readings.

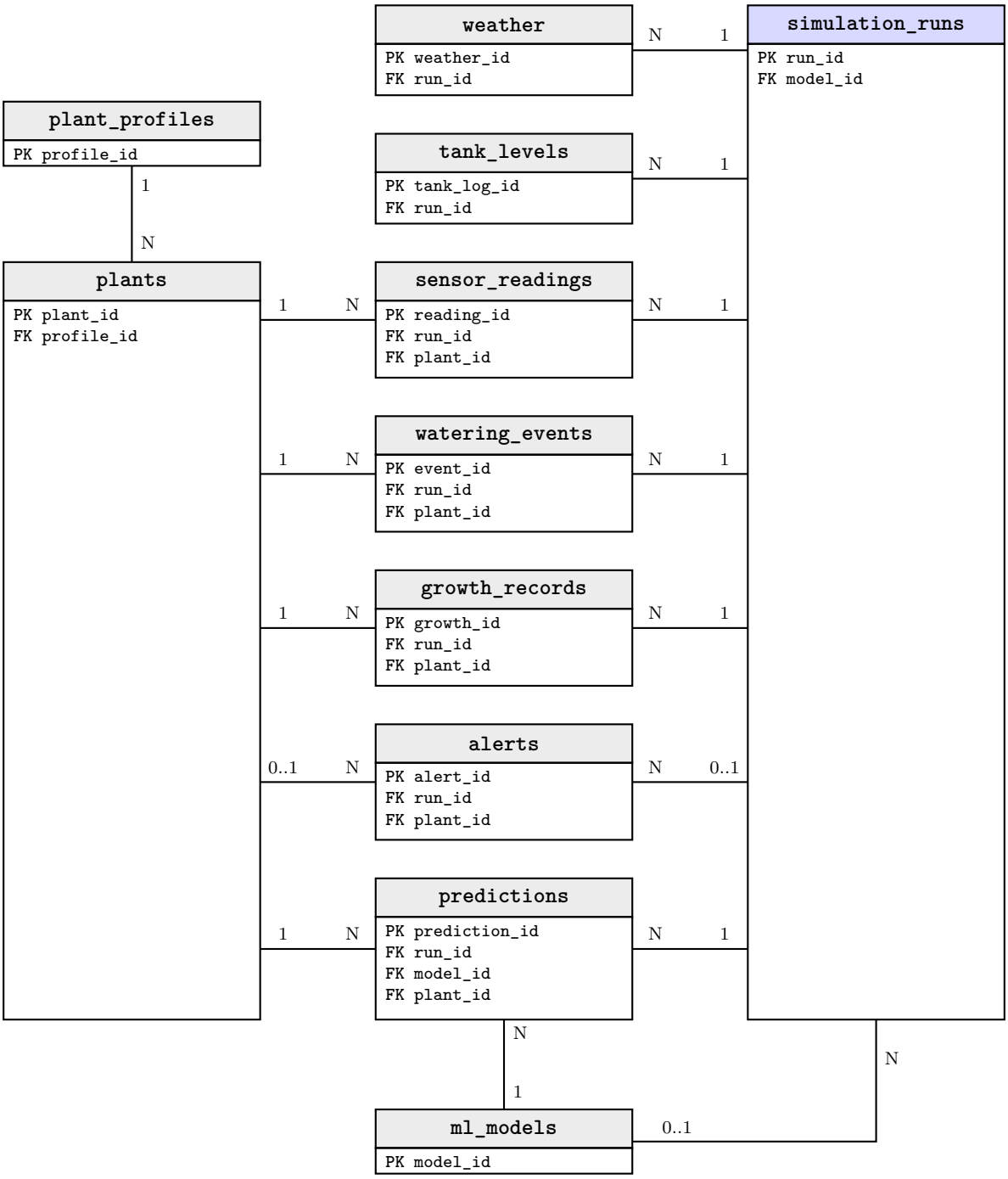


Figure 2: ER diagram of the waterX database. **1** = exactly one, **N** = many (zero or more), **0..1** = optional (nullable foreign key). The line goes from the parent (one) side to the child (many) side.